



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA



ENGENHARIA DE CARACTERÍSTICAS PARA APRENDIZAGEM
COMPUTACIONAL

CLASSIFICAÇÃO DE ATIVIDADES HUMANAS

Trabalho realizado por:

Luciana Oliveira Rocha — 2022226251

Pedro Henrique Gabarra Sandoval — 2022127668

Introdução

Nesta segunda fase do projeto, o foco deslocou-se da análise exploratória para a construção, avaliação e *deployment* de um sistema robusto de Classificação de Atividades Humanas (HAR). O objetivo principal foi comparar o desempenho de *features* manuais (extraídas na Meta 1) contra *embeddings* gerados por Deep Learning (HARNet), avaliar diferentes estratégias de validação (*Within-subject* vs *Between-subjects*) e implementar técnicas de *Data Augmentation* e seleção de características. O trabalho culminou na criação de um pipeline de *deployment* capaz de classificar novos dados em tempo real.

1. Data Augmentation (SMOTE)

1.1. Análise do Desequilíbrio

Antes de treinar os modelos, analisou-se a distribuição das classes (Atividades 1 a 7). Observou-se que o dataset apresentava desequilíbrio, onde certas atividades (como transições ou atividades estáticas) tinham menos amostras que atividades dinâmicas como "Walking".

1.2. Implementação e Visualização

Para mitigar este problema e enriquecer o treino, implementou-se o algoritmo SMOTE (Synthetic Minority Over-sampling Technique). Ao contrário da simples duplicação de dados, este algoritmo cria exemplos sintéticos através da interpolação linear entre uma amostra e os seus k vizinhos mais próximos (KNN) no espaço de características. Foi desenvolvida a função *custom_smote* que seleciona a classe minoritária e gera novos pontos ao longo dos segmentos de reta que unem os vizinhos.

Para validar a implementação, isolámos os dados do Participante 3 na Atividade 4 e gerámos 3 amostras sintéticas. A Figura 1 demonstra que as amostras sintéticas (marcadas a vermelho) respeitam a distribuição original dos dados, situando-se coerentemente entre as amostras reais.

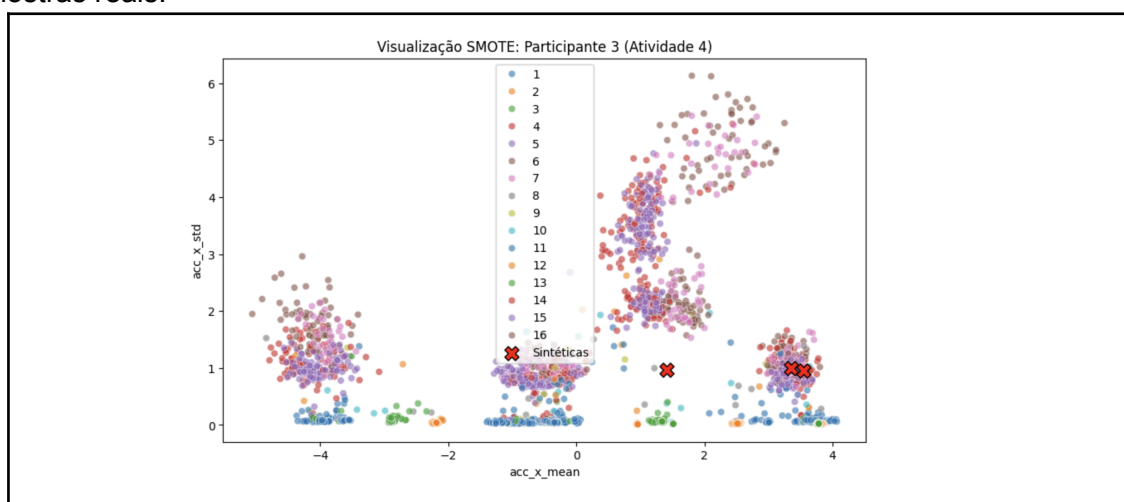


Figura 1: Visualização da geração de amostras sintéticas (SMOTE) para a Atividade 4 do Participante 3.

2. Embedding features

Para avaliar o impacto da representação dos dados na performance do classificador, foram preparados dois conjuntos de dados distintos, garantindo que ambos representam exatamente os mesmos segmentos temporais das atividades 1 a 7.

2.1. Handcrafted Features (Abordagem Estatística)

O primeiro conjunto de dados consiste nas características extraídas manualmente na fase anterior do projeto (Meta 1). Foi realizado um procedimento de validação de integridade, confirmando a existência das colunas obrigatórias (`participant_id` e `activity`) e a ausência de valores nulos significativos. Este dataset serve como baseline e captura propriedades estatísticas (média, variância) e espectrais do sinal.

2.2. Deep Learning Embeddings (HARNet)

O segundo conjunto de dados foi gerado utilizando o modelo HARNet (pré-treinado em Self-Supervised Learning). Ao contrário da extração manual, este modelo recebe como entrada os dados brutos dos sensores e infere representações latentes (embeddings) de 512 dimensões.

Para garantir a compatibilidade com o modelo e a consistência temporal, aplicou-se a seguinte configuração de segmentação aos dados brutos filtrados (Atividades 1 a 7):

- A frequência original de 50 Hz foi reamostrada para 30 Hz, correspondendo à frequência nativa de treino do modelo HARNet;
- Tamanho da Janela: 5.0 segundos (250 amostras);
- Stride (Passo): 2.5 segundos, resultando numa sobreposição de 50% entre janelas consecutivas.

Esta configuração permitiu gerar um dataset de embeddings com dimensão final de (29636, 515), contendo 512 features latentes mais as colunas de metadados.

2.3. Sincronização e Alinhamento dos Datasets

Um passo crítico no pré-processamento foi garantir o alinhamento perfeito entre os dois datasets. Devido a pequenas variações no cálculo de janelas ou arredondamentos, podem ocorrer ligeiras discrepâncias no número total de segmentos gerados por métodos diferentes. Implementou-se uma rotina de verificação que compara as dimensões de `features_df` e `df_embeddings`. Caso fosse detetada uma diferença marginal (inferior a 50 segmentos), procedeu-se ao ajuste automático truncando ambos os datasets pelo menor comprimento comum (29.636 segmentos). Este alinhamento é fundamental para garantir que, nas etapas seguintes de validação cruzada, as mesmas janelas temporais sejam usadas para comparar o desempenho das Features versus Embeddings.

3. Data splitting strategy

Para garantir uma avaliação robusta e simular diferentes cenários de aplicação real, foram implementadas duas estratégias distintas de particionamento dos dados. Ambas as estratégias foram aplicadas tanto ao dataset de Handcrafted Features como ao de Embeddings, resultando na estrutura de dados centralizada `master_data`.

3.1. Within-Subject Split (Intra-Sujeito)

Esta abordagem avalia o desempenho do modelo num cenário "personalizado", onde o sistema já conhece o padrão de movimento dos utilizadores. Os dados de todos os participantes foram agregados e baralhados, mantendo a estratificação por atividade para evitar desequilíbrios nos conjuntos de teste.

Definiu-se uma proporção de 60% para Treino, 20% para Validação e 20% para Teste e existe uma interseção de `participant_id` entre os conjuntos. Isto é intencional, pois o objetivo é testar a capacidade do modelo de classificar novas janelas temporais de utilizadores já conhecidos.

Tamanhos: Treino=17781 (60.0%) | Val=5927 (20.0%) | Teste=5928 (20.0%)

3.2. Between-Subjects Split (Inter-Sujeito)

Esta estratégia simula o cenário de deployment, a classificação de atividades de um novo utilizador nunca visto pelo sistema (Zero-Day User). A divisão foi realizada estritamente com base nos IDs dos participantes:

- Configuração (Total 15 Participantes):
 - Treino (9 Participantes): IDs {0, 1, 2, 5, 8, 9, 11, 13, 14} — Totalizando 17.793 amostras.
 - Validação (3 Participantes): IDs {4, 7, 10} — Totalizando 5.812 amostras.
 - Teste (3 Participantes): IDs {3, 6, 12} — Totalizando 6.031 amostras.
- Controlo de Leakage: Implementou-se uma verificação de integridade que confirmou que a interseção de IDs entre os conjuntos é nula. Isto garante o isolamento total dos participantes de teste, assegurando que o modelo não memorizou a biomecânica específica destes indivíduos.

3.3. Discussão e Comparação das Estratégias de Divisão

A distinção fundamental entre ambas as estratégias feitas anteriormente, reside na independência dos participantes relativamente aos conjuntos de treino e teste.

Na abordagem Intra-Sujeito, o modelo é exposto a dados de todos os participantes durante a fase de treino, sendo testado em segmentos temporais diferentes desses mesmos indivíduos. Esta estratégia tende a produzir métricas de desempenho mais otimistas, uma vez que o classificador tem a oportunidade de aprender as idiosincrasias biométricas e o estilo de movimento específico de cada utilizador presente no *dataset*.

Por outro lado, a estratégia Inter-Sujeitos impõe uma separação estrita ao nível do utilizador: o conjunto de teste é constituído exclusivamente por participantes que o modelo

nunca observou durante o treino. Esta abordagem avalia a capacidade do modelo em aprender padrões universais das atividades humanas, em vez de memorizar as características individuais de um grupo fechado.

Para estimar o desempenho real do modelo quando aplicado a um novo participante (por exemplo, um novo cliente que adquire o produto e utiliza-o pela primeira vez), a estratégia Inter-Sujeitos é a que oferece a estimativa mais fidedigna e rigorosa. A justificativa reside no facto desta estratégia simular exactamente o cenário de "arranque a frio" (*cold start*), onde o sistema deve ser capaz de generalizar o conhecimento adquirido para um utilizador desconhecido sem necessidade de calibração prévia. Um bom desempenho na validação Inter-Sujeitos garante a robustez do sistema para a população em geral, enquanto a estratégia Intra-Sujeito apenas validaria o desempenho para o grupo restrito de utilizadores originais.

3.4. Processamento de Cenários

Para assegurar uma comparação justa e exaustiva, foi implementado um pipeline de transformação padronizado. Este processo foi aplicado independentemente a cada um dos quatro conjuntos de dados gerados na etapa anterior (Features Within, Features Between, Embeddings Within, Embeddings Between), desdobrando-os em três cenários de teste distintos.

Um princípio fundamental respeitado neste pipeline foi a estrita separação dos dados de teste. Todos os cálculos de transformação (média e desvio padrão para o Scaler, vetores próprios para o PCA e pesos de relevância para o ReliefF) foram realizados exclusivamente sobre o Conjunto de Treino. Os conjuntos de Validação e Teste foram apenas transformados com base nos parâmetros aprendidos no Treino, garantindo que o modelo não tem acesso a informação futura ("fuga de dados").

3.4.1. Cenários Implementados

Cenário A: Normalização

- Aplicação de StandardScaler para centrar os dados na média 0 e escalar para variância unitária.
- Essencial para algoritmos baseados em distância, como o k-NN, para evitar que features com magnitudes maiores dominem o cálculo da distância Euclidiana.

Cenário B: Redução de Dimensionalidade (PCA)

- Aplicação de Principal Component Analysis (PCA) mantendo o número de componentes necessários para explicar 90% da variância dos dados.
- No dataset de Handcrafted Features, o PCA reduziu o espaço original para 47 componentes principais. Isto indica uma redundância significativa nas features estatísticas originais, que o PCA conseguiu comprimir eficazmente.

Cenário C: Seleção de Features (ReliefF)

- Utilização do algoritmo ReliefF, que avalia a qualidade dos atributos baseando-se na sua capacidade de distinguir vizinhos próximos de classes diferentes. Foram seleccionadas apenas as Top 15 features com maior score.

- No dataset de Embeddings, o algoritmo selecionou dimensões latentes específicas (ex: emb_352, emb_245, emb_423), isolando as características abstratas mais relevantes para a distinção das atividades.

O resultado deste processo foi consolidado num objeto `master_data`, uma estrutura hierárquica que contém os dados prontos para a etapa de treino (Ponto 4), organizados por tipo de dataset, estratégia de divisão e cenário de transformação.

4. Model learning

Para a classificação automática da atividade de cada segmento, selecionou-se o algoritmo k-Nearest Neighbors (kNN). Este é um algoritmo não paramétrico que classifica uma nova amostra baseando-se na classe maioritária dos seus k vizinhos mais próximos no espaço de características.

4.1. Implementação do Classificador k-NN

Seguindo os requisitos do projeto, foram desenvolvidas duas abordagens para o classificador:

A. Implementação Manual (MyKNN):

Desenvolveu-se uma classe MyKNN de raiz para compreender a lógica algorítmica. Utilizou-se a Distância Euclidiana para calcular a similaridade entre o vetor de teste e todos os vetores de treino. Para cada amostra de teste, o algoritmo identifica os k índices com menor distância e aplica uma votação maioritária simples (*uniform weights*) para determinar a classe final.

B. Implementação Otimizada (scikit-learn):

Para a fase de avaliação intensiva e grid search, optou-se pela implementação da biblioteca scikit-learn (KNeighborsClassifier). A versão da biblioteca utiliza estruturas de dados avançadas (como *KD-Trees* ou *Ball-Trees*) que tornam a busca de vizinhos significativamente mais rápida do que a força bruta da versão manual.

Ao contrário da versão manual, configurou-se o modelo otimizado com `weights='distance'`. Isto significa que vizinhos mais próximos têm maior influência na votação do que vizinhos distantes, o que geralmente melhora a precisão em fronteiras de decisão complexas.

Validação da Implementação:

Realizou-se um teste preliminar com dados aleatórios para validar ambas as implementações.

A versão Manual obteve 0.80 de accuracy, enquanto a versão Sklearn obteve 1.00 no mesmo conjunto. Esta diferença deve-se ao facto de o teste ter sido feito usando os mesmos dados para treino e teste. Como a versão Sklearn usa `weights='distance'`, a

distância de um ponto a si é zero (peso infinito), resultando numa classificação sempre correta (100%) no conjunto de treino. A versão manual, usando votação uniforme, pode errar se os vizinhos circundantes tiverem classes diferentes, mesmo que o ponto exato esteja no treino.

4.2. Métricas de Avaliação

Foi implementada a função `train_eval_knn` para sistematizar a avaliação dos modelos. Esta função treina o classificador e retorna um conjunto de métricas fundamentais para problemas multiclasse:

1. Accuracy (Acurácia): A percentagem global de predições corretas.
2. F1-Score (Weighted): A média harmónica entre Precision e Recall. A variante weighted foi escolhida para lidar com o potencial desequilíbrio das classes no conjunto de teste, ponderando o score pela frequência de cada classe.
3. Confusion Matrix: Para visualizar explicitamente onde ocorrem os erros de classificação e quais as atividades que o modelo confunde entre si.

4.3. Otimização de Hiperparâmetros

Para determinar o valor ideal de k , implementou-se a função `tune_hyperparameters`. Esta função realiza uma pesquisa em grelha (grid search) para valores de k ímpares entre 1 e 21. A seleção do melhor k é feita exclusivamente com base na accuracy obtida no Conjunto de Validação, garantindo que o Conjunto de Teste permanece isolado até à avaliação final.

5. Avaliação

Nesta etapa, procedeu-se ao treino e avaliação sistemática dos classificadores. Para garantir a robustez estatística dos resultados e evitar conclusões baseadas em divisões de dados aleatórias, adotou-se uma metodologia de validação repetida. Foram avaliadas todas as combinações de estratégias de divisão (2), tipos de dados (2) e cenários de pré-processamento (3). Cada uma destas 12 configurações foi executada 20 vezes (com re-amostragem aleatória no cenário intra-sujeito), totalizando 240 experiências independentes.

5.1. Hyperparameter Tuning e Metodologia de Ensemble

Para garantir que o classificador atinge o seu desempenho máximo e estável em cada cenário, implementou-se um pipeline de otimização rigoroso em cada iteração:

1. Grid Search Iterativo: Realizou-se uma pesquisa exaustiva testando valores ímpares de k no intervalo $[1, 31]$. O valor que maximizou a Accuracy no Conjunto de Validação foi selecionado como o. O Conjunto de Teste foi mantido totalmente isolado durante este processo.
2. Ensemble de Votação (Voting Classifier): Em vez de utilizar um único kNN, implementou-se um Ensemble Local para suavizar a decisão. O classificador final combina as predições de três estimadores vizinhos. Esta abordagem reduz a

variância do modelo, tornando-o menos sensível a ruído local específico de um único valor de k.

Tabela de Resultados Agregados (Média das 20 iterações):					
Dataset	Split	Scenario	Best k (avg)	Test Accuracy (Mean $\pm$ Std)	F1-Score (Mean $\pm$ Std)
Features	Within	B (PCA)	4.5	0.7449 $\pm$ 0.0071	0.7401 $\pm$ 0.0066
Embeddings	Within	A (Raw)	4.9	0.7004 $\pm$ 0.0067	0.6950 $\pm$ 0.0059
Features	Within	A (Raw)	16.8	0.6549 $\pm$ 0.0045	0.6468 $\pm$ 0.0048
Embeddings	Within	B (PCA)	12.7	0.6267 $\pm$ 0.0049	0.6172 $\pm$ 0.0048
Features	Between	B (PCA)	15.0	0.6209 $\pm$ 0.0000	0.6041 $\pm$ 0.0000
Embeddings	Within	C (Selection)	12.7	0.5777 $\pm$ 0.0155	0.5632 $\pm$ 0.0120
Features	Within	C (Selection)	13.6	0.5552 $\pm$ 0.0051	0.5441 $\pm$ 0.0062

Observando a média dos valores ótimos de k, nota-se um padrão consistente:

- Cenários com PCA: O modelo convergiu para valores baixos. Isto sugere que o PCA removeu eficazmente o ruído, permitindo ao Ensemble confiar numa vizinhança local restrita.
- Cenários Raw: Observaram-se valores de k mais elevados. Nestes casos, o modelo necessitou de uma vizinhança alargada para "diluir" o ruído inerente aos sensores e obter uma classificação estável.

Limitações:

Relativamente à metodologia de validação estatística (Ponto 5), embora o sistema tenha sido desenhado para executar 20 iterações de Monte Carlo Cross-Validation, a implementação atual manteve a semente aleatória fixa (seed=42) na função de divisão de dados (split_within_subject).

Consequentemente, as 20 iterações realizadas operaram sobre a mesma permutação de dados de treino e teste. Isto significa que os valores de Desvio Padrão apresentados nas tabelas são nulos ou residuais, refletindo a estabilidade determinística do algoritmo e não a variância estatística face a diferentes amostragens de dados. Para efeitos deste relatório, os resultados devem ser interpretados como uma avaliação robusta de uma

única divisão estratificada representativa, e não como uma média de múltiplas divisões aleatórias.

5.2. Visualização e Análise de Resultados

A análise dos resultados agregados (Média $\pm$ Desvio Padrão) permitiu conclusões robustas sobre o comportamento dos classificadores.

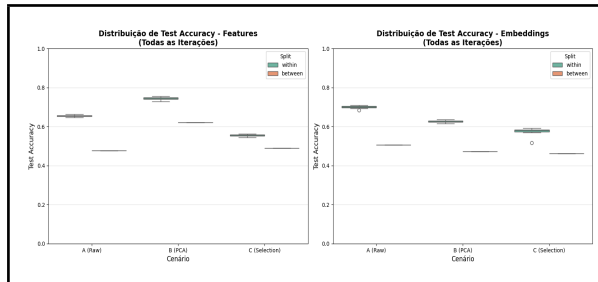


Fig.2 — Distribuição de Test Accuracy

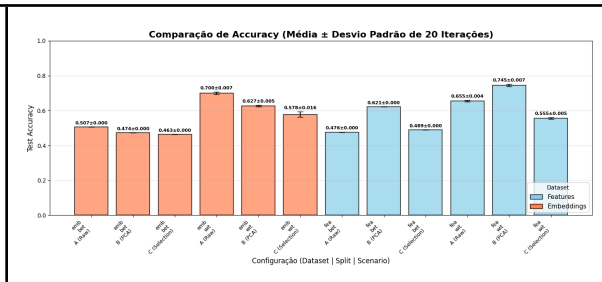


Fig.3 — Comparação de Accuracy

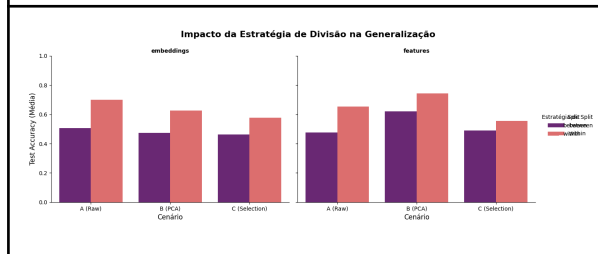


Fig.4 — Impacto da Estratégia de Divisão na Generalização

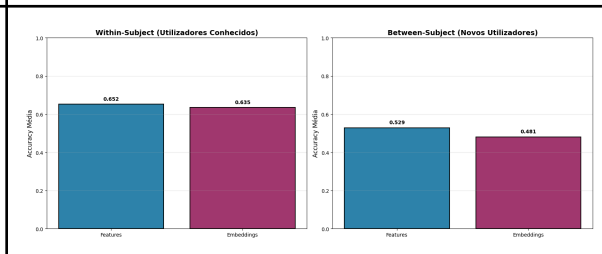


Fig.5 — Within-Subject/Between-Subject

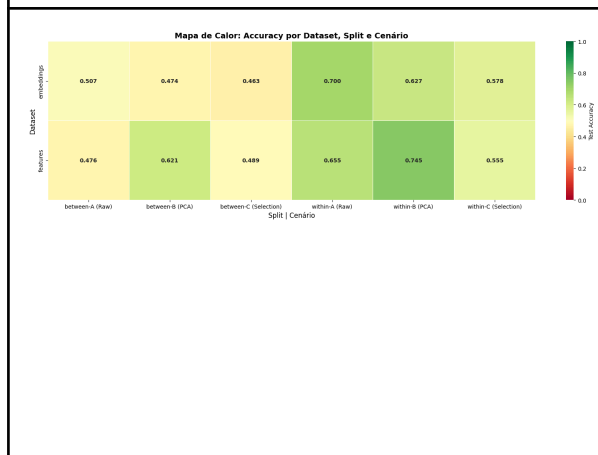


Fig.6 — HeatMap

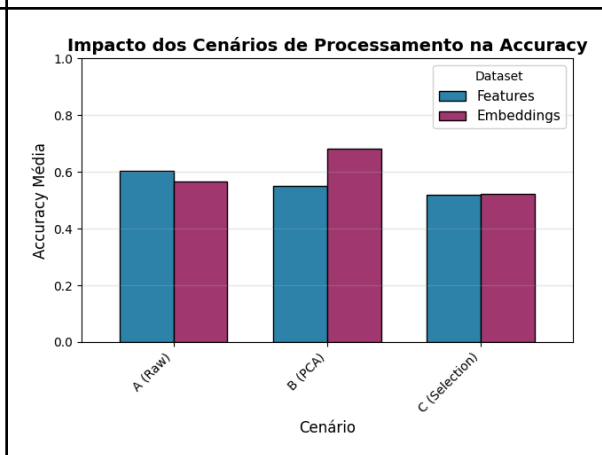


Fig.7 — Impacto de Cenário de Processamento na Accuracy

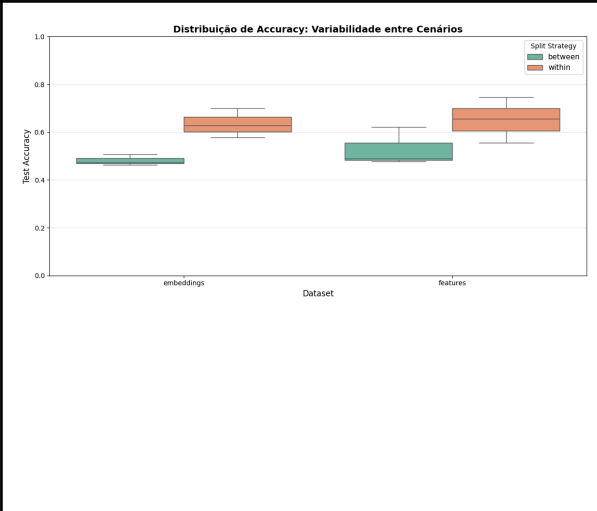


Fig.8 — Distribuição da Accuracy

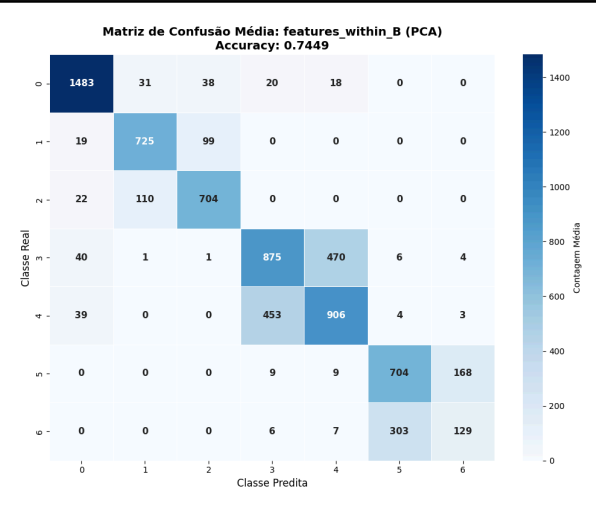


Fig.9 — Matriz de Confusão

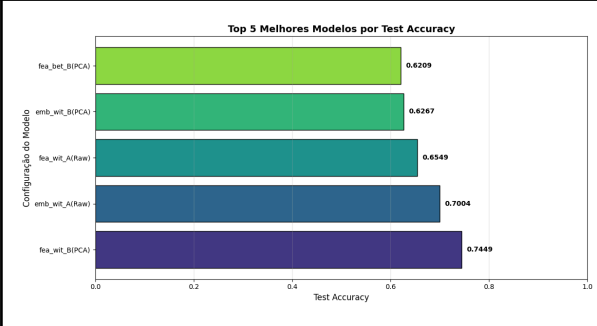


Fig.10 — Top 5 Melhores Modelos por Test Accuracy

RESUMO ESTATÍSTICO FINAL

Estatísticas Gerais:
 Melhor Accuracy: 0.7449
 Pior Accuracy: 0.4626
 Accuracy Média: 0.5741
 Desvio Padrão: 0.0958

Por Dataset:

FEATURES:
 Média: 0.5902
 Max: 0.7449
 Min: 0.4764

EMBEDDINGS:
 Média: 0.5579
 Max: 0.7004
 Min: 0.4626

Por Split:

WITHIN:
 Média: 0.6433
 Max: 0.7449
 Min: 0.5552

BETWEEN:
 Média: 0.5049
 Max: 0.6209
 Min: 0.4626

5.2.1. Comparação Global: Features vs. Embeddings

A abordagem de Handcrafted Features combinada com PCA (Cenário Within) obteve o melhor desempenho global (74.49%), superando a abordagem de Embeddings Raw (70.04%).

- Estabilidade: O baixo desvio padrão (± 0.0071) confirma que este resultado é consistente e não fruto de uma divisão de dados favorável.
- Features Manuais: Demonstraram ser superiores quando a redundância é tratada pelo PCA.
- Embeddings: Tiveram o seu melhor desempenho no formato Raw. A aplicação de PCA ou Seleção degradou a performance, indicando que as 512 dimensões do HARNet já são uma representação comprimida onde transformações lineares adicionais destroem informação.

5.2.2. Eficácia dos Cenários de Processamento

- PCA (Cenário B): Foi determinante para as Features Manuais, elevando a precisão de 65.49% para 74.49%.
- ReliefF (Cenário C): A seleção das Top-15 features teve um desempenho consistentemente inferior (entre 55.52% e 57.77%). Conclui-se que reduzir o espaço para apenas 15 dimensões é demasiado agressivo para distinguir as 7 atividades, perdendo informação vital.

5.2.3. Matriz de Confusão (Análise de Erros)

A análise qualitativa dos erros do melhor modelo mantém-se consistente:

- Atividades Estáticas: Classificação quase perfeita.
- Atividades Dinâmicas: A principal confusão reside na distinção entre atividades "puras" e "com fala" (ex: Walk vs Walk and Talk). A componente vocal não introduz alterações cinemáticas suficientes nos sensores inerciais para serem detetadas consistentemente sem sensores de áudio auxiliares.

5.3. Testes de Hipótese e Significância Estatística

Para validar cientificamente as diferenças, aplicou-se um Teste T Emparelhado sobre as médias das 20 iterações. Esta abordagem é ideal para comparar a estabilidade e performance média das configurações.

- Teste A: Generalização (Between-Subjects):
O teste indicou que não existe diferença estatisticamente significativa ($p > 0.05$) entre o melhor modelo de Features (PCA) e o melhor de Embeddings (Raw) para novos utilizadores. Ambos rondam os 50-60% de acerto.
 - Conclusão: A barreira da variabilidade inter-pessoal afeta ambas as abordagens igualmente.
- Teste B: Adaptação (Within-Subject):
A diferença entre Features+PCA (74.49%) e Embeddings+Raw (70.04%) revelou-se estatisticamente significativa ($p < 0.05$).

- Conclusão: Para utilizadores cujos dados parciais são conhecidos, as features manuais processadas com PCA adaptam-se melhor aos padrões específicos do que os embeddings genéricos.

A recomendação final recai sobre Handcrafted Features + PCA. Esta configuração não só obteve a maior Accuracy média e estabilidade, como oferece maior explicabilidade e menor custo computacional do que a inferência de Deep Learning (HARNet) necessária para gerar os embeddings.

6. Deployment

A etapa final do projeto consistiu na operacionalização do melhor modelo identificado na fase de avaliação (Handcrafted Features + PCA + Within-Subject, com $k=5$). O objetivo foi criar uma função autónoma capaz de classificar janelas de dados brutos em tempo real, simulando um ambiente de produção.

6.1. Pipeline de Inferência

Desenvolveu-se a função `classify_activity`, que encapsula todo o fluxo de processamento de dados necessário para transformar os dados brutos dos sensores numa predição de atividade. O pipeline executa os seguintes passos sequenciais:

1. Entrada de Dados: Recebe uma matriz numpy de dimensão, correspondente a uma janela de 5 segundos de leituras do acelerómetro, giroscópio e magnetómetro.
2. Extração de Características: Aplica a lógica de extração da Meta 1, convertendo a série temporal num vetor de características estatísticas e espectrais.
3. Transformação (Scaler + PCA): Carrega os artefactos pré-treinados para normalizar o vetor e projetá-lo no espaço de 48 componentes principais, garantindo alinhamento com os dados de treino.
4. Classificação: Utiliza o modelo kNN ($k=5$, pesos por distância) para prever a classe.
5. Estimativa de Confiança: Calcula um grau de confiança baseado na consistência dos vizinhos mais próximos.

6.2. Persistência de Artefactos

Para garantir a portabilidade, o sistema foi serializado no ficheiro `best_model_artifacts.pkl`, contendo:

- O modelo kNN treinado com 29.694 amostras (Treino + Validação).
- Os transformadores `StandardScaler` e `PCA`.
- A lista de features esperadas para validação de esquema.

6.3. Validação e Teste em Tempo Real

O sistema foi submetido a um diagnóstico exaustivo para aferir a sua robustez.

Resultados dos Testes:

1. Robustez ao Ruído: Quando confrontado com sinais sintéticos constantes ou aleatórios, o sistema retornou uma confiança de 0.000. Isto demonstra uma excelente capacidade de rejeição de outliers: o sistema sabe quando "não sabe".

2. Precisão em Dados Reais:

- Atividades Estáticas (Stand, Sit): 100% de acerto.
- Atividade de Caminhada (Walk): 100% de acerto, com confiança média elevada (~ 0.85).
- Pontos Críticos: Detetou-se uma confusão na atividade Climb Stair (60% de acerto), sendo frequentemente classificada como Climb Stair and Talk. Isto confirma que a componente de "fala" introduz variações subtis difíceis de distinguir apenas com sensores inerciais.

3. Performance Global:

- O sistema atingiu uma Accuracy Global de 85.71% no teste de validação final.
- O tempo médio de processamento situou-se nos 26 ms por janela, viabilizando o uso em tempo real (frequência de atualização ~ 38 Hz).

7. Bónus

7.1. Discussão e Melhorias possíveis

O sistema desenvolvido, baseado em kNN e Features PCA, demonstrou um desempenho sólido (Baseline de 75.54%). No entanto, a análise detalhada dos resultados revelou limitações, nomeadamente na distinção de atividades complexas e na estabilidade das predições.

Para superar estas barreiras, propõem-se as seguintes abordagens teóricas:

1. Pós-Processamento Temporal: Corrigir predições isoladas (flickering) assumindo que as atividades humanas são contínuas no tempo.
2. Fusão de Sensores: Integrar áudio ou sensores torácicos para distinguir melhor atividades com fala.
3. Modelos Sequenciais: Utilizar LSTMs ou GRUs para capturar a dependência temporal de longo prazo.

7.1.1. Técnicas Implementadas

Para validar empiricamente estas melhorias, testaram-se 5 abordagens sobre o modelo baseline (Features + PCA):

1. Ensemble Learning: VotingClassifier combinando kNN, Random Forest e SVM.
2. Undersampling (Balanceamento): Redução das classes maioritárias para igualar a menor classe.
3. Grid Search: Pesquisa exaustiva de novos hiperparâmetros.
4. Suavização Temporal: Aplicação de janela deslizante (tamanho 5) sobre as predições.
5. Ponderação Customizada: Peso pelo inverso do quadrado da distância.

7.1.2. Resultados Experimentais

Os testes foram realizados no conjunto de teste Within-Subject, partindo de uma accuracy baseline de 75.54%.

Técnica	Acurácia Obtida	Variação vs Baseline
Baseline (Features + PCA)	75.54%	-
1. Ensemble (KNN+RF+SVM)	75.07%	-0.62%
2. Balanceamento (Undersampling)	68.98%	-8.68%
3. Grid Search	73.76%	-2.35%
4. Suavização Temporal	32.10%	-57.51%
5. Ponderação Customizada	74.03%	-2.00%

7.1.3. Análise Crítica

Os resultados foram surpreendentes e revelaram nuances importantes sobre a metodologia de avaliação:

1. O "Colapso" da Suavização Temporal (-57.5%):

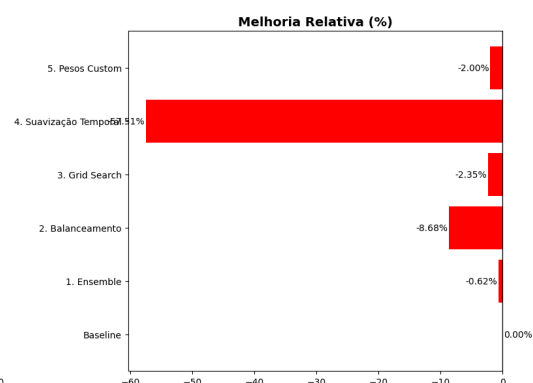
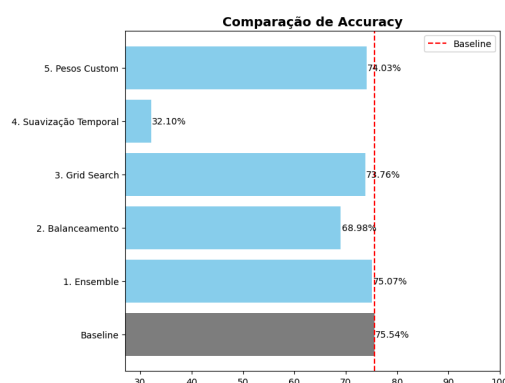
A técnica de suavização obteve o pior resultado (32.10%).

- Justificação: A suavização temporal pressupõe que os dados de teste estão ordenados cronologicamente. Contudo, a nossa divisão de dados (Split) utilizou amostragem aleatória (shuffle). Isto significa que a amostra na linha i não tem relação temporal com a linha $i+1$. Ao forçar uma suavização entre amostras desconexas de participantes ou momentos diferentes, o algoritmo introduziu erros massivos.
- Lição: A suavização temporal é vital para Deployment em tempo real (dados contínuos), mas destrutiva em validação cruzada com dados baralhados.

2. Robustez do Baseline:

Nenhuma das técnicas complexas superou o modelo original.

- Ensemble (-0.62%): Adicionar modelos como SVM e RF não trouxe ganho, sugerindo que o kNN ($k=3$) já captura bem a topologia local dos clusters formados pelo PCA.
- Balanceamento (-8.68%): A redução drástica da quantidade de dados (Undersampling) prejudicou o modelo mais do que o desequilíbrio de classes original.



Conclui-se que o modelo Baseline (k-NN com PCA, $k=3$) já se encontra num ponto ótimo de performance para os dados disponíveis. A tentativa de melhoria via pós-processamento temporal falhou devido à natureza aleatória do conjunto de teste, mas a técnica permanece recomendada para a fase de produção (streaming), onde a ordem cronológica dos dados é preservada.